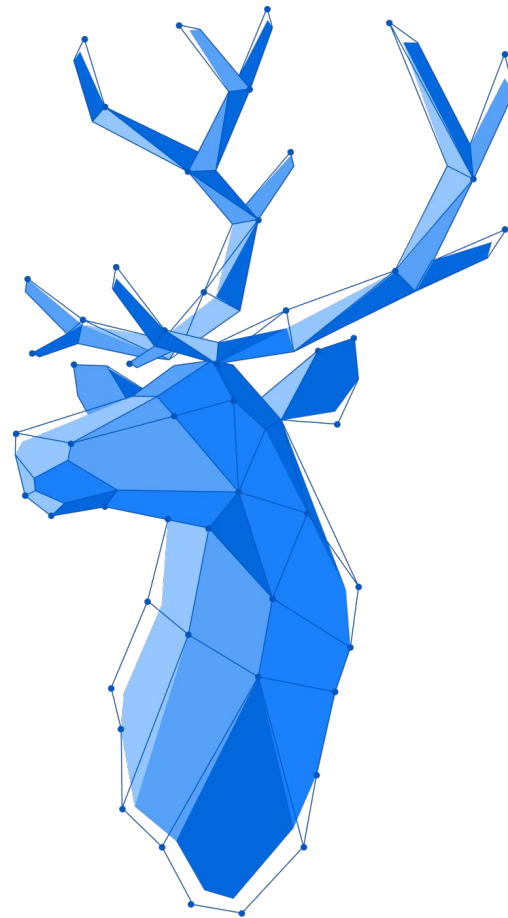




Dependency Injection

By Gerald Koch



Spring Bean Lifecycle



Instantiate



(Re)Use



Collect

- Manages and instantiates `@Component/@Bean` (and extending) annotated classes and provides them
- Dependencies are resolved via dependency tree
- Calling constructors should be avoided (spring instantiates classes). That helps avoiding memory leaks
- Static constructor parameters need to be applied vis setter
- Beans may have different scopes

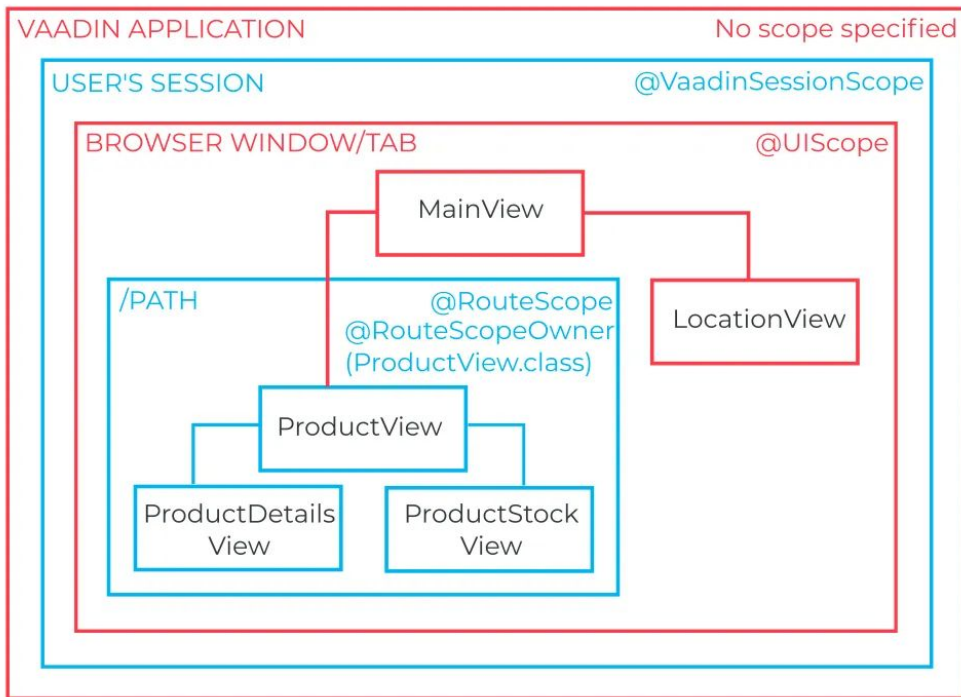
Dependency Injection

Spring - Scopes

- Singleton Scope (only one instance)
 - No additional scope annotation necessary
 - `@Scope("singleton")`
 - `@Scope(value = ConfigurableBeanFactory.SCOPE_SINGLETON)`
- Prototype Scope (every injection creates a new instance)
 - `@Scope("prototype")`
 - `@Scope(value = ConfigurableBeanFactory.SCOPE_PROTOTYPE)`
- Session Scope (one instance per session)
 - `@SessionScope`
 - `@Scope(value = WebApplicationContext.SCOPE_SESSION, proxyMode = ScopedProxyMode.TARGET_CLASS)`
- More Scopes: <https://docs.spring.io/spring-framework/reference/core/beans/factory-scopes.html>

Dependency Injection

Vaadin - Scopes



- Vaadin Application (no Scope) corresponds to the singleton scope
- @VaadinSessionScope lies withing @SessionScope

More information:

<https://vaadin.com/docs/latest/flow/integrations/spring/scopes>

Dependency Injection

Hierarchy of Scopes

```
@UIScope //one per Vaadin UI
public class User {

    private String name;
    private String email;
    private String address;
}
```

```
@Service //singleton scope
public class UserService {

    private User user;

    public Presenter (User user) { // which one?
        this.user = user;
    }
}
```

- Smaller scopes can easily access bigger scopes over injections
- Bigger scopes need support to choose the right instance of a smaller scope class (except for instances of scope prototype)
- Proxy are necessary for injections of classes with smaller Scopes into classes of bigger scopes

Scopes Proxies

- Proxies enable bigger scopes to access classes of smaller scopes
- Proxies are created automatically from the Spring framework , if configured
- The type of proxy which is to be used is set using the property `proxyMode`
 - `@Scope(value = XYZ, proxyMode = ScopedProxyMode.TARGET_CLASS)`
- There are different proxy modes:
 - **DEFAULT:** Uses the globally configured proxy mode
 - **NO:** Does not use any proxy
 - **TARGET_CLASS:** Uses the CGLIB library to inherit a proxy class from the original class
 - **INTERFACES:** uses the injected interface to create a proxy class

Attention: Only interfaces are allowed to be used for injection
- Vaadin scopes don't have proxies configured
- Spring scopes, which are not resulting in singleton, have `TARGET_CLASS` configured

Dependency Injection

Spring - Value Injection

- `@Value(<value id>)`
 - Can be used with the Property Placeholder Syntax `"${...}"`
 - Can be used with Spring Expression Language (SpEL) `"#{${...}}"`
- Property Placeholder Syntax:
 - Without default value: `${property}`
 - With default value: `${property:default}`
 - Works with String and primitive data types and their object wrappers
 - Works with lists of above mentioned types*: `(prop=value1,value2,value3)`
- Spring Expression Language
 - Without default (value mandatory): `#{${property}}`
 - With default (value optional): `#{${property:default}}`
 - Supports maps `(prop={key1:'value1', key2:'value2', ...})`

It's always better with a default value

Spring – Best Practices

- Stay as stateless as possible (reduce class variables to a minimum) especially in the repository and service layer
- Try always to inject into the constructor and avoid `@Autowired` at class variables (prohibits cyclic references and memory leaks)
- UI components: create layout and componentes inside the constructor and use methods like `setItems(...)` or `setDataProvider(...)` to set data (see Vaadin class `Grid`)
- Do not use Spring in a static context
- Avoid instantiation via constructors! Spring creates instances. If you use constructors to create your instances, you will lose the Spring context
- Use defaults on `@Value` annotations if possible